



Semana 4 - Revisão

Problema A - 2D Array

Simple problema de implementação.

Complexidade: $O(1)$

```
#include "stdio.h"
#include "stdlib.h"

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int mat[6][6];
    for(int i = 0; i < 6; i++) {
        for(int j = 0; j < 6; j++) scanf("%d",&mat[i][j]);
    }

    int ans = -0x3f3f3f3f;
    for(int i = 0; i <= 3; i++) {
        for(int j = 0; j <= 3; j++) {
            int cima = mat[i][j]+mat[i][j+1]+mat[i][j+2];
            int meio = mat[i+1][j+1];
            int baixo = mat[i+2][j]+mat[i+2][j+1]+mat[i+2][j+2];
            ans = max(ans, cima+meio+baixo);
        }
    }

    printf("%d\n",ans);
}
```

Problema B - Arrays: Left Rotation

Ao invés de simularmos as rotações, podemos utilizar aritmética modular. Dado que os elementos serão shiftados d índices à esquerda, sabemos que um elemento no índice i ($0 \leq i \leq n-1$) estará no índice $(i-d) \pmod n$, ou em outras palavras, no índice $(i-d+n) \% n$.

Complexidade: $O(n)$

```
#include "stdio.h"
#include "stdlib.h"

int main() {
    int n,d; scanf("%d-%d",&n,&d);
    int v[n];
    for(int i = 0; i < n; i++) scanf("%d",&v[i]);

    int ans[n];
    for(int i = 0; i < n; i++) ans[(i-d+n)%n] = v[i];
}
```



```
    for(int i = 0; i < n; i++) printf("%d-",ans[i]);  
    printf("\n");  
}
```

Problema C - Divisible Sum Pairs

Pelo fato do n ser pequeno, podemos testar todos os pares (i, j) de maneira quadrática.

Complexidade: $O(n^2)$

```
#include "stdio.h"  
#include "stdlib.h"  
  
int main() {  
    int n,k; scanf("%d-%d",&n,&k);  
    int v[n];  
    for(int i = 0; i < n; i++) scanf("%d",&v[i]);  
  
    int ans = 0;  
    for(int i = 0; i < n; i++) {  
        for(int j = i+1; j < n; j++) ans += ((v[i]+v[j])%k == 0);  
    }  
  
    printf("%d\n",ans);  
}
```

Problema D - CamelCase

Segundo o problema, para cada palavra subsequente a primeira letra é sempre maiúscula. Logo, para determinar a quantidade de palavras no total, basta contar a quantidade de letras maiúsculas que aparecem na string em camelCase.

Para isso, analisaremos cada caractere da string e verificamos seu código ASCII. O alfabeto maiúsculo está entre os códigos 65 e 90, sendo eles os códigos de 'A' e 'Z', respectivamente.

Complexidade: $O(|s|)$

```
#include "stdio.h"  
#include "stdlib.h"  
  
int main() {  
    char s[200000]; scanf("%s",s);  
  
    int ans = 1, idx = 0;  
    while(s[idx] != '\0') {  
        if(s[idx] <= 90) ans++;  
        idx++;  
    }  
  
    printf("%d\n",ans);  
}
```

Problema D - Helpful Maths

Podemos resolver este problema por apenas contando a quantidades de 1's, 2's e 3's que aparecem na string. Uma vez todos contados, basta imprí-los em ordem: primeiro os 1's, em seguida os 2's e por os 3's, separados por '+'.



Complexidade: $O(|s|)$

```
#include "stdio.h"
#include "stdlib.h"

int main() {
    char s[200001]; scanf("%s", s);
    int uns = 0, dois = 0, tres = 0;

    for(int i = 0; s[i] != '\0'; i++) {
        if(s[i] == '+') continue;
        else if(s[i] == '1') uns++;
        else if(s[i] == '2') dois++;
        else tres++;
    }

    int total = uns + dois + tres;
    while(total > 1) {
        if(uns) {
            printf("1+");
            uns--;
            total--;
            continue;
        }

        if(dois) {
            printf("2+");
            dois--;
            total--;
            continue;
        }

        if(tres) {
            printf("3+");
            tres--;
            total--;
        }
    }

    if(uns) printf("1\n");
    else if(dois) printf("2\n");
    else printf("3\n");
}
```